TP 07 — Système de fichiers

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	—
CO2	Understand process scheduling, memory management, and file system organization algorithms	✓
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	—

Contexte

Vous êtes experte en analyse d'artefacts numériques : on vous remet le fichier de synthèse `evidence.txt` et un espace de travail. Mission en trois objectifs : **métadonnées** et **accès direct** ; **copie, lien physique et lien symbolique** ; **espace disque**. Chaque objectif se termine par un **script interactif** qui valide votre travail et génère un rapport.

Tableau de mission

Objectif	Mission	Script	Récompense
0	Préparer le poste : déployer l'outil `mkoslabs`	— (PDF)	5 pts
1	Documenter les métadonnées et l'accès direct	`lab07_ex1.sh`	5 pts
2	Copies, lien physique et lien symbolique	`lab07_ex2.sh`	5 pts
3	Espace disque du dossier de mission	`lab07_ex3.sh`	5 pts

Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

* Interpréter les métadonnées d'un fichier et distinguer nom et inode
* Lire un bloc précis d'un fichier en accès direct
* Distinguer copie, lien physique et lien symbolique au niveau de l'inode
* Expliquer l'effet d'une suppression (compteur de liens, lien cassé)
* Mesurer l'espace occupé et l'espace libre, et relier les écarts à l'allocation par blocs

Pré-requis et matériel

* Terminal Bash fonctionnel (WSL2 sur Windows, Terminal.app sur macOS, ou Linux)
* Avoir assisté au CMs et au TPs précédents.
* L'archive `scripts_lab07.zip`, récupérée par `mkoslabs` (Exercice 0)
* Le fichier de données `evidence.txt`, fourni dans l'archive

Déroulement et barème

3 objectifs de 25 minutes, précédés de l'**Exercice 0** (30 minutes) où vous déployez l'outil `mkoslabs` fourni. Chaque objectif se lance par son script (`./lab07_exN.sh`) : il guide, vérifie l'état de votre machine, donne des indices, puis génère un **rapport** (`report_lab07_exN.txt`). Ouvrez **deux terminaux** dans le répertoire du TP : l'un pour le script, l'autre pour le travail.

Évaluation par synthèse manuscrite. À la fin de chaque objectif, rédigez sur papier ce que vous avez fait, pourquoi ça marche et le lien avec le cours.

Barème (sur 20) : Exercice 0 « Environnement prêt » (5 pts) + trois objectifs (5 pts chacun). Machine non opérationnelle = 0/20.

Annexe — Aide-mémoire des commandes du TP

Ce tableau ne remplace pas le cours. En cas de doute, utilisez `man commande` ou `commande --help`.

Fichiers, métadonnées et liens

Commande	Syntaxe	Rôle
`ls -l`	`ls -l [chemin]`	Détail : type, permissions, liens, propriétaire, taille, date
`ls -i`	`ls -i fichier`	Affiche le numéro d'inode
`ls -li`	`ls -li dossier/`	Inode en tête, puis le détail de chaque entrée
`ls -lc`	`ls -lc fichier`	Affiche l'horodatage de changement (ctime)
`cp`	`cp source cible`	Copie ; si la cible est un répertoire `dossier/`, le nom est conservé
`ln`	`ln cible nouveau_nom`	Lien physique : second nom du **même** inode
`ln -s`	`ln -s cible lien`	Lien symbolique : petit fichier contenant un chemin (inode propre)
`wc`	`wc [-l] [-c] fichier`	Compte lignes (`-l`) ou octets (`-c`)
`mkdir`	`mkdir -p chemin`	Crée un répertoire (+ parents manquants)

Un lien physique ne traverse pas les systèmes de fichiers et reste interdit sur les répertoires ; un lien symbolique peut viser un répertoire ou un autre système de fichiers, mais devient cassé *si sa cible disparaît.*

Accès direct et espace disque

Commande	Syntaxe	Rôle
`printf`	`printf '%s\n' a b c`	Écrit une ligne par argument (format réutilisé)
`dd`	`dd if=f bs=8 skip=3 count=1`	Lit un bloc précis : taille `bs`, saut `skip`, nombre `count`
`dd` (écriture)	`dd of=f bs=8 seek=2 conv=notrunc`	Écrit au bloc `seek` ; `conv=notrunc` empêche la troncature
`df -k`	`df -k [chemin]`	Espace des systèmes de fichiers, en blocs de 1 Ko
`du -s`	`du -k -s dossier`	Total seul pour un répertoire
`command -v`	`command -v nom`	Indique si une commande est disponible dans le `PATH`

`dd` affiche des lignes « records in/out » sur la sortie d'erreur : c'est normal. Dans ce TP, chaque enregistrement fait 8 octets (7 caractères + retour à la ligne) : `bs` doit donc valoir 8. `df -T` (type de système de fichiers) est GNU et absent de macOS ; `du -k` peut varier d'un système de fichiers à l'autre, c'est pourquoi le script recalcule la valeur sur votre machine.

Aide, scripts et vérification

Commande	Syntaxe	Rôle
`man`	`man commande`	Page de manuel complète
`man -k`	`man -k mot-clé`	Recherche une commande par mot-clé
`--help`	`commande --help`	Aide rapide
`grep`	`man commande | grep -e "-option"`	Chercher une option dans le manuel
`micro`	`micro fichier.sh`	Éditeur de texte du cours
`chmod`	`chmod +x script.sh`	Rend un script exécutable
`source`	`source ~/.bashrc`	Recharge la config. (zsh macOS : `~/.zshrc`)
`export PATH`	`export PATH="$HOME/bin:$PATH"`	Ajoute `~/bin` au chemin de recherche des commandes (`$PATH`)

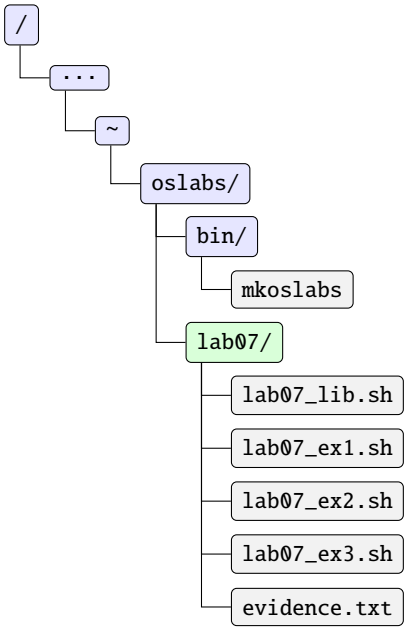
Un motif commençant par un tiret est pris pour une option : `grep "-l"` échoue (`-l` = option de `grep`). Formes sûres : `grep -e "-l"`, `grep -- "-l"` ou `grep "[-]l"`. `man -k` s'appuie sur `whatis` : si vide, `sudo mandb` (Linux) ou `sudo makewhatis` (macOS).

1 Exercice 0 — Déployer l'outil de préparation

Durée : 30 min — 5 points « Environnement prêt »

Contexte

Avant toute mission, on prépare son poste. Aux TPs précédents, ces commandes se recopiaient à la main ; cette fois, un outil prêt à l'emploi — `mkoslabs` — est publié sur la **page du TP**. Votre travail : créer le fichier dans `~/oslabs/bin`, y coller son contenu, le rendre exécutable, ajouter le dossier au `PATH`, puis le lancer avec une option et l'argument propre à ce TP. L'outil sera réutilisé tel quel aux TPs suivants (`mkoslabs -i 08`). À la fin de cet exercice, votre arborescence doit ressembler à ceci :



Instructions

1. Aller à la racine : `cd /`
2. Revenir au répertoire personnel : `cd ~`
3. Entrer dans le dossier commun : `cd oslabs`
4. Créer le dossier des outils : `mkdir -p bin`
5. Entrer dans le dossier des outils : `cd bin`
6. Créer le fichier de l'outil : `nano mkoslabs` (ou `micro mkoslabs` si installé)
7. Ouvrir la **page du TP07**, copier le contenu du fichier `mkoslabs`, le coller dans l'éditeur, enregistrer et quitter
8. Rendre l'outil exécutable : `chmod +x mkoslabs`
9. Revenir au répertoire personnel : `cd`
10. Identifier son shell : `echo $SHELL`
    - **WSL (bash)** : le fichier de configuration est `~/.bashrc`

- **macOS (zsh)** : le fichier de configuration est `~/.zshrc`

11. Ajouter le dossier des outils au `PATH` :
    - **WSL** : `echo 'export PATH="$HOME/oslabs/bin:$PATH"' >> ~/.bashrc`
    - **macOS** : `echo 'export PATH="$HOME/oslabs/bin:$PATH"' >> ~/.zshrc`
12. Recharger la configuration du shell :
    - **WSL** : `source ~/.bashrc`
    - **macOS** : `source ~/.zshrc`
13. Vérifier que l'outil est trouvé : `command -v mkoslabs`
14. Afficher son aide : `mkoslabs -h`
15. Vérifier les outils nécessaires : `command -v curl unzip dd df du`
16. Préparer le TP avec l'option d'installation et l'argument du TP : `mkoslabs -i 07`
17. Entrer dans le dossier du TP : `cd ~/oslabs/lab07`
18. Vérifier le contenu du dossier : `ls`

Attendu : vous trouvez les 5 fichiers du schéma — `lab07_lib.sh`, `lab07_ex1.sh`, `lab07_ex2.sh`, `lab07_ex3.sh` et `evidence.txt` — ainsi que `scripts_lab07.zip`. Le dossier `dossier/` sera créé pendant l'objectif 1.

Le fichier `mkoslabs` est publié sur la page du TP : recopiez-le à l'identique (aucune modification). Ses options sont `-i` (installation des paquets manquants) et `-h` (aide) ; le numéro du TP est l'argument, sur deux chiffres.

`curl -O` est disponible par défaut sur Linux, macOS et WSL. Sous macOS (zsh), le fichier de configuration est `~/.zshrc` ; sous WSL, c'est `~/.bashrc`.

Appelez le professeur pour valider votre environnement avant de commencer les objectifs.

2 Objectif 1 — Métadonnées et accès direct

Durée : 25 min — 5 points

Documenter les métadonnées d'une pièce (type, inode, taille, horodatages), en créer des copies de travail et découper un fichier d'enregistrements pour relire un bloc précis dans un ordre libre.

Lancez le script (terminal 1) : `./lab07_ex1.sh`

3 Objectif 2 — Copie, lien physique et lien symbolique

Durée : 25 min — 5 points

Créer deux noms pour un même fichier, vérifier ce qu'ils partagent réellement, puis comparer avec un lien vers un chemin, en observant les effets d'une suppression sur chacun.

Lancez le script (terminal 1) : `./lab07_ex2.sh`

4 Objectif 3 — Espace disque du dossier de mission

Durée : 25 min — 5 points

Mesurer l'espace occupé par les dossiers de travail et l'espace libre des systèmes de fichiers, enregistrer un relevé et relier les écarts à l'allocation par blocs.

Lancez le script (terminal 1) : `./lab07_ex3.sh`

Appelez le professeur avec vos rapports `report_lab07_exN.txt` en main, accompagnés d'une synthèse libre manuscrite (notes personnelles) : ce que vous avez fait, pourquoi ça marche, et le lien avec le cours.